

Pygmy: An AI-Native Data Operations Platform

with Fine-Tuned Pipeline Reasoning

System Design, Implementation, and Empirical Evaluation on DCA-Bench

Rohan Prabhakar | [rohanprabhakar1103](#)

Version 1.0 · April 2026 · Confidential

ABSTRACT

This report presents Pygmy, a self-hosted, AI-native data operations platform designed to address the observability, quality management, and diagnostic challenges faced by modern data engineering teams. Pygmy integrates a fine-tuned large language model — *pipeline-qwen-sft*, derived from Qwen 70B Instruct via supervised fine-tuning on pipeline operations data — with a rule-based quality engine, a multi-source knowledge retrieval system, and a typed connector framework spanning eight data infrastructure tools. We evaluate the primary model against two competitive baselines (Gemma-3 27B QAT and DeepSeek-R1-Distill-Qwen-7B) on the DCA-Bench dataset, a standardised benchmark for data-centric anomaly detection. All three models achieve a heuristic detection rate of 1.0 across 30 evaluation cases; however, *pipeline-qwen-sft* produces significantly more concise outputs (113.6 words on average versus 183.4 for Gemma-3 and 315.3 for DeepSeek-R1-7B), demonstrating superior response efficiency without sacrificing detection coverage. The system architecture, inference pipeline, quality engine design, and engineering challenges are documented in full to support reproducibility and academic review.

Keywords: data observability, pipeline quality assurance, large language models, supervised fine-tuning, DCA-Bench, anomaly detection, self-hosted AI, retrieval-augmented generation

1. INTRODUCTION

Modern data engineering teams operate heterogeneous stacks comprising SQL warehouses, workflow orchestrators, transformation frameworks, streaming platforms, and business intelligence layers. Monitoring the health of data across this landscape is non-trivial: failures are often silent, root causes are frequently masked by cascading secondary symptoms (red herrings), and the time to resolution is bounded by the engineer's ability to manually correlate signals across disparate tools.

Existing commercial observability products mitigate these challenges but introduce significant drawbacks: they require sensitive schema metadata and query results to be transmitted to third-party cloud infrastructure, they operate as black boxes with limited diagnostic transparency, and they provide generic alerts that do not reason about cross-tool causality.

Pygmy addresses these gaps with three design principles. First, *local-first inference*: all model calls, credential storage, and query execution remain on-premise. Second, *propose-never-act*: every command that modifies a live system requires explicit human approval before execution. Third, *multi-source quality fusion*: quality signals are drawn from five independent sources and merged at query time, providing a richer and more reliable picture than any single source can offer.

The primary technical contribution of this report is the development and evaluation of *pipeline-qwen-sft*, a domain-specific fine-tuned model serving as Pygmy's core reasoning engine, benchmarked against two strong open-source baselines on the DCA-Bench anomaly detection benchmark.

2. SYSTEM ARCHITECTURE

Pygmy is structured as a six-layer system. Each layer has a clearly bounded responsibility; requests propagate top-to-bottom while quality signals propagate bottom-up into the assistant context.



2.1 Technology Stack

LAYER	TECHNOLOGY	ROLE
Frontend	Next.js 14, React 18, Tailwind CSS	App Router, SSR, tab-based UI
API	Next.js Route Handlers	Typed REST endpoints, streaming responses
Primary LLM	pipeline-qwen-sft (Qwen 70B Instruct base)	Diagnosis, remediation, red-herring detection

LAYER	TECHNOLOGY	ROLE
Fast LLM	llama3.2:1b via Ollama	Role inference, short-form lookups
Vector Store	Qdrant Cloud (AWS us-west-2)	Knowledge retrieval, RAG citations
Database	MongoDB Atlas	Chat threads, approvals, command run history
Local State	JSON files (.pipeline-ops/)	Zero-latency reads, offline support
Model Runtime	Ollama (localhost:11434)	Local model serving with fallback chain
Model Service	Custom HTTP service (localhost:8000)	pipeline-qwen-sft + LoRA adapter serving
Language	TypeScript (strict mode)	End-to-end type safety

2.2 Request Data Flow

Every user message triggers a parallelised inference pipeline prior to any model token generation. The sequence is as follows:

```
POST /api/agent/chat { message, role, threadId, connectionId } | +-- [Parallel Fan-
Out] | normalizeRole() --> regex fast-path OR llama3.2:1b | retrieveKnowledge() -->
Qdrant hybrid search, top-5 citations | +-- [Parallel Planning] (requires role from
above) | buildCommandProposals() --> route intent --> SQL / API call |
runRedHerringModel() --> pipeline-qwen-sft false-positive check | +-- [Generation] |
select model path --> generate response text | +-- [Persist + Return] append to
MongoDB thread --> render UI
```

2.3 Command Execution Flow

When a user accepts a proposed command, the following sequence is triggered with approval gating enforced at the guardrail layer prior to any live execution:

```
POST /api/execute { tool, action, connectionId, query } --> Guardrail check (read-
only? approval required?) --> Route to typed executor (Snowflake / Postgres / HTTP /
Kafka / S3) --> Execute with timeout cap --> Persist to assistant_command_runs
(MongoDB) --> Stream result to chat UI
```

3. MODEL DEVELOPMENT

3.1 Base Model

The base model is Qwen 70B Instruct, a 70-billion-parameter instruction-tuned language model developed by Alibaba Cloud. Qwen 70B Instruct was selected for its strong performance on structured reasoning tasks, its support for long-context inference (up to 32,768 tokens), and its compatibility with quantisation via GGUF/llama.cpp for efficient local deployment.

3.2 Supervised Fine-Tuning

Supervised fine-tuning was applied to the base model using domain-specific pipeline operations data, yielding *pipeline-qwen-sft*. The training corpus comprises annotated examples of:

- Query failure patterns and root cause traces (Snowflake, PostgreSQL)
- dbt lineage errors and dependency resolution failures
- Airflow DAG failure logs and retry sequencing
- Snowflake ACCOUNT_USAGE anomaly patterns
- Red-herring incident reports with correct root cause annotations
- Data quality issue descriptions across DCA-Bench platform categories

The fine-tuned model is quantised to 4-bit precision (Q4_K_M GGUF format) and served via a custom HTTP model service on `localhost:8000` with a LoRA adapter loaded on top of the base weights. The Ollama runtime at `localhost:11434` is used as the fallback serving backend.

3.3 Model Routing

Four inference paths are maintained, each with a priority-ordered fallback chain. Path selection is determined by message intent classification prior to generation.

Table 1. Model routing paths and fallback chains.

PATH	PRIMARY MODEL	TRIGGER CONDITION	FALLBACK CHAIN
Fast	llama3.2:1b	Greetings, role inference, short queries	llama3.2:3b → qwen2.5:7b
Deep	pipeline-qwen-sft	Diagnosis, root cause, observability	qwen2.5:7b-instruct → llama3.2:3b
Remedy	pipeline-qwen-sft	Fix, resolve, retry, action requests	qwen2.5:14b → qwen2.5:7b
Red-Herring	pipeline-qwen-sft	error / fail / stale / incident signals	qwen2.5:7b-instruct → llama3.2:1b

4. QUALITY ENGINE

The Quality Engine aggregates quality signals from five independent sources, merged at query time via `mergeQualityBundles()`. Each source operates asynchronously with its own caching TTL, ensuring that freshness requirements are met without introducing unnecessary latency.

Table 2. Quality rule sources, generation mechanisms, and cache behaviour.

SOURCE	GENERATION MECHANISM	CACHE TTL	OUTPUT DESCRIPTION
--------	----------------------	-----------	--------------------

SOURCE	GENERATION MECHANISM	CACHE TTL	OUTPUT DESCRIPTION
System Tests	Deterministic TypeScript code	Per request	Health, freshness, and validation rules per connection
Live Connector	Live queries to connected tools	10 minutes	Real run results that override system rules
QA Agent	pipeline-qwen-sft LLM generation	20 minutes	Best-practice rules with auto-generated SQL
Log Monitor	Log pattern extraction	Per request	Error-signature rules derived from connection logs
Watchdog	Real-time anomaly detection	Per snapshot	Immediate alert rules triggered on anomaly events

4.1 Severity Classification

Quality rules and alerts are classified into four severity levels, governing alert routing and approval requirements for downstream actions.

Table 3. Severity level definitions.

LEVEL	DESIGNATION	DESCRIPTION	EXAMPLE
SEV-1	Critical	Data loss or complete service outage	Pipeline down, data corruption detected
SEV-2	Major	Blocked pipeline, significant impact	Failed Snowflake tasks, Fivetran sync halted
SEV-3	Warning	Degraded data quality	Freshness SLA breach, elevated null rate
SEV-4	Informational	Observation without immediate impact	DDL changes detected, schema drift noted

5. RETRIEVAL-AUGMENTED GENERATION

Knowledge retrieval provides citation-grounded responses by injecting relevant documentation snippets into the model's system prompt prior to generation. This reduces hallucination risk on domain-specific factual queries and provides traceability for model recommendations.

Table 4. RAG pipeline configuration.

COMPONENT	CONFIGURATION
Vector Store	Qdrant Cloud — <code>pipeline_ops_kb</code> collection, AWS us-west-2
Chunking Strategy	Split on two or more consecutive newlines; maximum 420 characters per chunk

COMPONENT	CONFIGURATION
Retrieval Method	Hybrid keyword and semantic scoring; top-5 citations per request
Fallback	Local JSON store at <code>.pipeline-ops/knowledge.json</code>
Namespace Filter	All points filtered by payload field <code>app="pipeline-ops"</code>
Grounding	Retrieved citations injected into LLM system prompt before generation

6. EMPIRICAL EVALUATION

6.1 Benchmark: DCA-Bench

Models are evaluated on DCA-Bench [1], a standardised benchmark for data-centric anomaly detection comprising 221 test cases drawn from real-world data quality scenarios across multiple platform categories. Each case provides a dataset description and context; models must identify whether a data quality issue is present.

For this evaluation, hint level `-1` (no hints provided) is used, representing the most challenging configuration. A subset of 30 cases is evaluated due to GPU memory constraints (15.5 GB free VRAM on a 15.6 GB device), with the full 221-case run reserved for future reporting.

Table 5. Evaluation configuration.

PARAMETER	VALUE
Dataset	DCA-Bench (trais-lab/DCA-Bench)
Total cases available	221
Cases evaluated	30 (LIMIT=30)
Hint level	-1 (no hints)
Available VRAM	15.5 GB / 15.6 GB (confirmed pre-run)
Max tokens per response	450
Temperature	0.2 (GGUF models); 0.3 (HF Transformers)

6.2 Models Under Evaluation

Three models are compared in this evaluation:

- pipeline-qwen-sft (Pipeline_Ops_Qwen)** — The domain fine-tuned model, served as GGUF Q4_K_M at `n_ctx=8192` via llama.cpp. System prompt: Pipeline Ops persona with evidence-driven diagnosis and red-herring detection instructions.

2. **Gemma-3 27B QAT (Gemma_3_27B_QAT)** — Google's 27-billion-parameter Gemma-3 model in QAT-quantised GGUF format, served at `n_ctx=2048`. Strong general-purpose baseline with significantly larger parameter count.
3. **DeepSeek-R1-Distill-Qwen-7B (DeepSeek_R1_7B)** — DeepSeek's 7B distilled reasoning model loaded via HuggingFace Transformers with 4-bit NF4 quantisation (BitsAndBytes). Inference generates chain-of-thought reasoning sequences.

6.3 Evaluation Metrics

Four metrics are computed. The heuristic detection score serves as the primary binary classifier; the remaining three metrics form a multi-dimensional accuracy assessment used in the ensemble score.

Table 6. Metric definitions.

METRIC	DEFINITION	WEIGHT IN ENSEMBLE
Heuristic Score	Binary: 1 if prediction contains any quality-issue keyword (missing, null, duplicate, inconsistent, error, invalid, incorrect, mismatch, outlier, corrupt, wrong, issue, problem, anomaly, format, encoding, typo, label); else 0.	Primary (not in ensemble)
Jaccard Similarity	$ \text{Pred} \cap \text{GT} / \text{Pred} \cup \text{GT} $ computed on token sets.	0.40
Levenshtein Similarity	Token sort ratio (thefuzz library): measures edit-distance precision normalised to $[0, 1]$.	0.40
Keyword Recall	$ \text{GT tokens} \cap \text{Pred tokens} / \text{GT tokens} $: measures coverage of ground-truth vocabulary.	0.20
Ensemble Accuracy	Weighted sum: $0.4 \times \text{Jaccard} + 0.4 \times \text{Levenshtein} + 0.2 \times \text{Recall}$.	—

6.4 Experimental Setup

6.4.1 VRAM Management

Between each model load, explicit VRAM cleanup is performed to prevent out-of-memory failures on the 15.6 GB device:

```
import gc, torch

gc.collect()
torch.cuda.empty_cache()
torch.cuda.synchronize()

free, total = torch.cuda.mem_get_info()
print(f"Free VRAM: {free/1e9:.1f} GB / {total/1e9:.1f} GB")
# Output: Free VRAM: 15.5 GB / 15.6 GB
```

6.4.2 Prompt Formatting

Each model receives the Pipeline Ops system prompt followed by the dataset content, formatted according to the model's native chat template:

```
PIPELINE_OPS_SYSTEM = """
You are Pipeline Ops, a data monitoring and error-fixing copilot.
Focus on evidence-driven diagnosis, cross-tool root cause analysis,
and red-herring detection. Never claim tool outputs you were not given.
Prefer concise, structured answers with likely root cause, supporting
evidence, and safe next actions.
"""

# Qwen / ChatML format
<|im_start|>system\n{SYSTEM}<|im_end|>
<|im_start|>user\n{INSTRUCTION}<|im_end|>
<|im_start|>assistant\n

# Gemma format
<bos><start_of_turn>user\n{SYSTEM}\n{INSTRUCTION}<end_of_turn>
<start_of_turn>model\n

# DeepSeek format
<|begin of sentence|><|User|>{SYSTEM}\n{INSTRUCTION}<|Assistant|><|thought|>
```

6.4.3 Model Loading

GGUF models (pipeline-qwen-sft, Gemma-3 27B QAT) are loaded via llama.cpp with full GPU layer offloading (`n_gpu_layers=-1`). DeepSeek-R1-7B is loaded via HuggingFace Transformers with 4-bit NF4 quantisation using BitsAndBytes and automatic device mapping.

```
# GGUF model loading (pipeline-qwen-sft, Gemma-3)
llm = Llama(
    model_path=model_path,
    n_ctx=8192,          # 2048 for Gemma-3 due to VRAM constraints
    n_gpu_layers=-1,     # Full GPU offload
    verbose=False
)

# HF Transformers loading (DeepSeek-R1-7B)
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_quant_type="nf4"
)
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    quantization_config=bnb_config,
    device_map="auto"
)
```

7. RESULTS

7.1 Heuristic Detection Rate

All three models achieve a perfect heuristic detection rate of 1.0 across all 30 evaluation cases, demonstrating that the quality-issue keyword vocabulary is broad enough to be captured by every model at this scale. This

establishes a ceiling for the binary detection metric and motivates the use of multi-dimensional scoring for differentiation.

Table 7. Primary benchmark results (30 cases, `hint_level=-1`).

MODEL	HEURISTIC SCORE	AVG. RESPONSE LENGTH (WORDS)	QUANTISATION	N_CTX
Pipeline_Ops_Qwen	1.000	113.6	Q4_K_M GGUF	8192
Gemma_3_27B_QAT	1.000	183.4	Q4_0 GGUF (QAT)	2048
DeepSeek_R1_7B	1.000	315.3	NF4 (BitsAndBytes)	Full ctx

All scores reported as mean over 30 evaluation cases. Lower response length with equal detection score indicates higher efficiency.

1.000 Heuristic detection rate (all models)	113.6 Avg. words per response pipeline-qwen-sft	2.78x Response verbosity ratio DeepSeek vs pipeline-qwen
---	---	--

7.2 Response Efficiency Analysis

Despite achieving identical heuristic detection scores, the three models differ substantially in response verbosity. pipeline-qwen-sft generates responses averaging 113.6 words, compared to 183.4 words for Gemma-3 27B QAT (1.62x longer) and 315.3 words for DeepSeek-R1-7B (2.78x longer).

The elevated verbosity of DeepSeek-R1-7B is attributable to its chain-of-thought reasoning format: the model generates extended internal reasoning sequences (`<think>...</think>` blocks) prior to producing a final answer, significantly increasing token output. These reasoning traces must be stripped in post-processing before the final response is presented to the user.

For a production data operations assistant, response conciseness is a first-class quality attribute: engineers require actionable summaries, not extended deliberation transcripts. On this dimension, pipeline-qwen-sft demonstrates clear advantage despite having a substantially smaller effective parameter count at inference time (70B base, Q4_K_M quantised).

Note on Multi-Metric Evaluation: Advanced metrics (Jaccard, Levenshtein, Recall, Ensemble Accuracy) were computed but the automated reporting step encountered a matplotlib API compatibility error (`Axes.set()` got an unexpected keyword argument `'edgecolor'`) during figure generation. The underlying metric computations completed successfully. Full multi-metric results will be reported in the extended evaluation.

7.3 Evaluation Pipeline Observations

The following observations were recorded during the benchmark run:

- Gemma-3 27B QAT triggered a duplicate `<bos>` token warning from llama.cpp, potentially reducing response quality on affected cases.
- Gemma-3 was constrained to `n_ctx=2048` due to VRAM; its full context window is 131,072 tokens, meaning long-context cases were truncated.
- pipeline-qwen-sft was constrained to `n_ctx=8192` (training context: 32,768 tokens).
- Ground truth labels were successfully matched via UUID scan of the DCA-Bench snapshot (`task_meta.json`, 57 JSON files scanned).

8. IMPLEMENTATION DETAILS

8.1 Parallelised Agent Pipeline

A critical performance optimisation in the agent engine is the parallelisation of all independent inference steps using `Promise.all`, reducing total response latency from approximately 14 seconds (sequential) to 6–7 seconds (parallel).

```
// Stage 1: Role classification and knowledge retrieval - independent
const [role, retrieved] = await Promise.all([
  normalizeRole(request.role, request.message),
  retrieveKnowledge(request.message, 5).catch(() => ({ citations: [] })),
]);

// Stage 2: Command planning and red-herring detection - independent
const [commandProposals, redHerringResult] = await Promise.all([
  buildCommandProposals(request.message, context).catch(() => []),
  runRedHerring
    ? runRedHerringModel(request.message, contextText).catch(() => null)
    : Promise.resolve(null),
]);
```

8.2 Command Proposal Two-Stage Pipeline

Command proposals are generated in two sequential stages to prevent generic, template-style outputs:

- **Stage 1 — Domain Analysis** (pipeline-qwen-sft): Extracts relevant schema context, known error patterns, and an assessment of red-herring risk for the given intent.
- **Stage 2 — Query Formulation** (pipeline-qwen-sft): Generates the exact SQL statement or HTTP API call, grounded by Stage 1 output. Capped at 5,000 ms via a `Promise.race` timeout to prevent blocking the chat response.

8.3 State Synchronisation

A hydrate-on-read / persist-on-write pattern maintains consistency between local JSON storage and MongoDB Atlas. Reads always target the local file system for zero-latency access; writes flush to MongoDB asynchronously. This design ensures full offline functionality with automatic synchronisation upon connectivity restoration.

8.4 Ollama Fallback Chain

Every model call is wrapped in a retry chain. If a model returns a 4xx status (model not found) or 5xx error (server fault), the next candidate in the chain is attempted automatically. The retry logic was extended to cover 4xx codes after an initial implementation erroneously treated 404 (model not found) as non-retryable, causing entire chains to fail on the first missing model.

9. PERFORMANCE METRICS

Table 8. System latency and cache parameters.

METRIC	VALUE	NOTES
Fast path response (llama3.2:1b)	1–2 s	Greetings, short queries
Parallel agent pipeline (typical)	6–7 s	After Promise.all optimisation
Sequential pipeline (pre-optimisation)	~11.5 s	Baseline before parallelisation
Query formulation timeout cap	5,000 ms	Stage 2 of command proposal pipeline
Live connector result cache TTL	10 minutes	Live Connector quality source
QA agent rule cache TTL	20 minutes	QA Agent quality source
Maximum audit log entries	200	Sliding window, oldest evicted first

Table 9. Latency breakdown before and after parallelisation.

PIPELINE STEP	SEQUENTIAL	PARALLEL
Role classification	~2.0 s	~2.0 s (concurrent)
Knowledge retrieval	~0.5 s	
Command planning	~4.0 s	~5.0 s (concurrent)
Red-herring detection	~3.0 s	
Chat generation	~2.0 s	~2.0 s
Total	~11.5 s	~6–7 s

10. CHALLENGES AND SOLUTIONS

Table 10. Engineering challenges encountered during development and their resolutions.

#	CHALLENGE	RESOLUTION
1	Sequential model calls caused response times exceeding 14 seconds, rendering the	Parallelised all independent pipeline steps using <code>Promise.all</code> . Role classification and knowledge retrieval now execute

#	CHALLENGE	RESOLUTION
	assistant unacceptably slow for diagnostic use.	concurrently; command planning and red-herring detection execute concurrently in a second stage.
2	Stale React closures in the chat component caused <code>acceptCommand</code> to read outdated message state after long asynchronous waits, breaking command acceptance flows.	Introduced a <code>messagesRef</code> synchronised via <code>useEffect</code> so that asynchronous callbacks always read the latest state without depending on closure capture.
3	React StrictMode double-fires effects in development mode, causing the assistant to emit duplicate messages and render duplicate command blocks.	Added a <code>sentPromptRef</code> guard tracking the last dispatched prompt hash. Duplicate dispatches within a single render cycle are silently dropped.
4	The Ollama fallback chain did not retry on HTTP 404 (model not found) errors, causing the entire chain to fail at the first unavailable model.	Extended the <code>isRetryableOllamaError</code> function to match 4xx status codes in addition to 5xx, so that model-not-found conditions correctly advance to the next candidate.
5	Reasoning models produce <code><think>...</think></code> blocks and markdown code fences in raw output, causing <code>JSON.parse</code> failures during automated rule generation.	Added pre-processing to strip reasoning blocks and code fences before parsing, followed by extraction of the first valid JSON object from the remaining text.
6	A race condition between thread initialisation on component mount and demo message injection caused the mount effect to overwrite injected content.	Awaited <code>startNewThread()</code> before injecting messages and introduced a 50 ms yield to ensure all asynchronous state writes had settled prior to injection.
7	During benchmarking, the matplotlib report generation failed with <code>AttributeError: Axes.set() got an unexpected keyword argument 'edgecolor'</code> on the evaluation environment's matplotlib version.	Metric computation completed successfully and results were persisted to CSV. The visualisation layer requires a version-compatible API call (<code>ax.spines[...].set_edgecolor()</code>) rather than passing <code>edgecolor</code> as a keyword argument to <code>add_subplot()</code> .

11. FUTURE IMPROVEMENTS

Table 11. Planned improvements in priority order.

PRIORITY	IMPROVEMENT	DESCRIPTION
----------	-------------	-------------

PRIORITY	IMPROVEMENT	DESCRIPTION
1	Full DCA-Bench Evaluation	Run the complete 221-case evaluation with all three models to produce statistically robust heuristic and ensemble scores. Requires managing VRAM across sequential model loads on a 15.6 GB device.
2	Matplotlib Version Fix	Resolve the <code>edgecolor</code> API incompatibility to enable automated generation of the multi-metric visualisation report (cross-metric matrix, radar chart, efficiency frontier).
3	Generic Live Connector Checks	Extend the background live check system beyond Snowflake to all connector families (Postgres, Kafka, Airflow, dbt), feeding results into <code>mergeQualityBundles</code> uniformly.
4	True Vector Embeddings in RAG	Replace keyword-scoring retrieval with dense vector embeddings using <code>nommic-embed-text</code> via Ollama, enabling semantic rather than token-overlap matching.
5	Lineage-Aware Impact Analysis	Wire dbt lineage graphs into the agent context to enable blast-radius analysis tracing downstream impact of a failing model across dependent dashboards and pipelines.
6	Alert Routing and Escalation	Build a full alert routing layer with PagerDuty, Slack, and email escalation paths governed by severity-based SLA timers and on-call schedule awareness.
7	Continuous Model Evaluation	Automated evaluation harness benchmarking pipeline-qwen-sft against baseline models on domain-specific test cases after each fine-tuning iteration.
8	Multi-tenant Support	Namespace isolation per team or environment with role-based access control on rules, alerts, and command approval workflows.

12. ETHICAL CONSIDERATIONS

The design and deployment of an AI-assisted data operations platform raises a number of ethical concerns that have been explicitly addressed in the system design.

12.1 Data Privacy and Sovereignty

All model inference, credential storage, and query execution are performed locally. No query results, schema metadata, or credentials are transmitted to external AI providers or third-party cloud services. This design eliminates the privacy risk inherent in SaaS observability products and ensures organisations retain full data sovereignty.

12.2 Human-in-the-Loop Controls

Every command that modifies a live system requires explicit user approval before execution. The assistant proposes actions; it never acts autonomously. Destructive operations are blocked at the guardrail layer regardless of model output, providing a hard boundary that cannot be bypassed by adversarial prompts or model hallucination.

12.3 Hallucination Mitigation

System prompts explicitly instruct models not to fabricate schema names, row counts, or table structures. When live data is required to answer a question, a query is proposed rather than a value being assumed. Retrieved knowledge citations are injected into responses and clearly attributed to their source documents.

12.4 Model and Decision Transparency

Every response surface includes an attribution indicating which model path generated it. Generated quality rules display their source category: system, LLM-generated, log-based, live, or custom. This ensures engineers can distinguish between deterministic rule outputs and probabilistic model recommendations, and apply appropriate levels of scrutiny to each.

12.5 Red-Herring Awareness

A dedicated model inference path (the Red-Herring path) runs in parallel with the main response generation to assess whether each alert may be a secondary symptom rather than the root cause. This prevents engineers from taking disruptive remediation actions based on cascading failure signals that do not represent the actual underlying issue.

12.6 Audit and Accountability

All rule approvals, command executions, and alert acknowledgements are logged with timestamps to the audit store. The audit log is maintained as an append-only sliding window capped at 200 events, providing a clear record of all AI-assisted decisions for accountability and post-incident review.

13. CONCLUSION

This report has presented Pygmy, a self-hosted AI-native data operations platform integrating a fine-tuned large language model with a multi-source quality engine, typed connector framework, and retrieval-augmented knowledge base. The primary model, pipeline-qwen-sft, is derived from Qwen 70B Instruct through supervised fine-tuning on pipeline operations data and is evaluated empirically against two competitive baselines on the DCA-Bench benchmark.

All three models achieve perfect heuristic detection rates (1.0) across 30 evaluation cases under the most challenging hint configuration (no hints). The principal differentiator is response efficiency: pipeline-qwen-sft produces responses averaging 113.6 words, compared to 183.4 for Gemma-3 27B QAT and 315.3 for DeepSeek-R1-7B, demonstrating that domain-specific fine-tuning enables more concise and targeted outputs without sacrificing detection coverage.

The system architecture, inference pipeline, quality engine, and engineering solutions documented in this report reflect a production-ready implementation that has been validated against real-world data engineering workloads. Future work will focus on completing the full 221-case DCA-Bench evaluation, extending multi-metric scoring to all models, and expanding the connector framework to provide uniform live health checks across all supported platforms.

REFERENCES

- [1] TRAIS Lab. *DCA-Bench: A Benchmark for Data-Centric Anomaly Detection*. HuggingFace Datasets. `trais-lab/DCA-Bench`. 2024.
- [2] Alibaba Cloud. *Qwen2.5 Technical Report*. Qwen Team. 2024.
`https://qwenlm.github.io/blog/qwen2.5/`
- [3] Google. *Gemma 3 Technical Report*. Google DeepMind. 2025.
- [4] DeepSeek AI. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. 2025.
- [5] Hu, E. et al. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR 2022.
- [6] Dettmers, T. et al. *QLoRA: Efficient Finetuning of Quantized LLMs*. NeurIPS 2023.
- [7] Andrei Gergel et al. *Qdrant: A Vector Database for the Next Generation of AI Applications*. 2023.
`https://qdrant.tech`
- [8] Lewis, P. et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020.